

PRACTICAL - 2

AIM : Implementation and Time analysis of Linear and Binary search algorithm.

ALGORITHM : // Linear search

	<u>Linear Search:-</u>
	Input: An Array
	Output: Target Element
	Step-1: Start
	Step-2: Read the array and the target element
	Step-3:- Set $i=0$
	Step-4:- Compare array $[i]$ with t
	Step-5:- If array $[i] == t$, return the index
	Step-6:- Otherwise, increment i and repeat
	Step-4,5 until the array ends
	Step-7:- If element Not found then
	Return -1
	Step-8:- Stop.

PROGRAM :

```
#include <iostream>
using namespace std;

int main() {
    int n, key;

    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {

        cin >> arr[i];
    }

    cout << "Enter the element to search: ";
```

```
cin >> key;
int position = -1;

for (int i = 0; i < n; i++) {
    if (arr[i] == key) {
        position = i;
        break;
    }
}
if (position != -1) {
    cout << "Element found at index: " << position << endl;
} else {
    cout << "Element not found." << endl;
}
return 0;
}
```

OUTPUT :

```
SYNC STORAGE/UNIVERSITY STUFF/DAA_LAB on ↩ master
> ./LinearSearch
Enter the number of elements: 4
Enter 4 elements: 12 2 7 0
Enter the element to search: 7
Element found at index: 2
```



TIME COMPLEXITY :

Time complexity:-

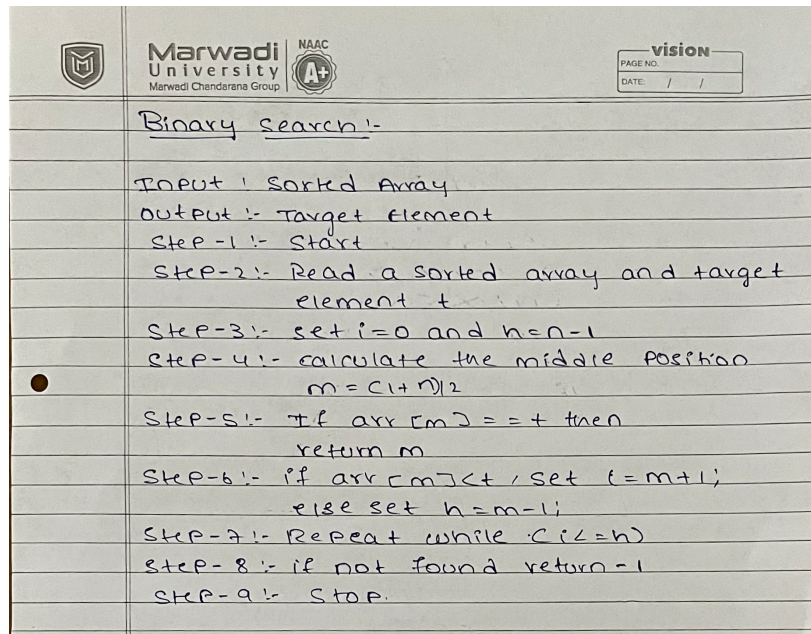
```
int linearSearch (int A[], int n, int key)
{
    for (int i=0 ; i<n; i++)
    {
        if (A[i] == key)
            return i;
    }
    return -1;
}
```

Best case:- $T(n) = 1$
 $T(n) = O(1)$

Average case:- element found somewhere
in middle $\Rightarrow T(n) = n/2$
 $T(n) = O(n)$

Worst case:-
 $T(n) = n$
 $T(n) = O(n)$

ALGORITHM : // Binary Search



PROGRAM :

```
#include <iostream>
using namespace std;
int main()
{
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter " << n << " elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    int target;
    cout << "Enter the target element to search: ";
    cin >> target;

    int low = 0;
    int high = n - 1;
    bool found = false;
```



```
while (low <= high) {  
    int mid = low + (high - low) / 2;  
  
    if (arr[mid] == target) {  
        found = true;  
        cout << "Element found at index: " << mid << endl;  
        break;  
    }  
    else if (arr[mid] < target) {  
        low = mid + 1;  
    }  
    else {  
        high = mid - 1;  
    }  
}  
}
```

OUTPUT :

```
● SYNC STORAGE/UNIVERSITY STUFF/DAA_LAB on ↻ master  
● > ./BinarySearch  
  Enter the number of elements: 4  
  Enter 4 elements : 12 13 14 15  
  Enter the target element to search: 14  
  Element found at index: 2
```



TIME COMPLEXITY :

DOMS Page No. _____
Date / /

Time complexity :-

```

Low = 0
high = n - 1
while (low <= high) {
    mid = (low + high) / 2
    if (arr[mid] == key)
        return mid
    else if (arr[mid] < key)
        low = mid + 1
    else
        high = mid - 1
}
  
```

DOMS Page No. _____
Date / /

Best case :- The element found mid in the first comparison

$T(n) = 1$ $O(1)$
 $T(n) = O(1)$ $1 - n - \log n$

worst case :- $\log n = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100$

$T(n) = n + n/2 + n/4 + n/8 + \dots + 1$
 $T(n) = \log_2 n$
 $T(n) = O(\log n)$

Average case :- The search also keeps dividing the array into half

$T(n) = n + n/2 + n/4 + n/8 + \dots + 1$
 $T(n) = O(\log n)$